

网络攻防技术复习资料

1. 第一章 网络攻击概述

1.1 了解

网络安全威胁分为广义和狭义：

广义网络安全威胁：任何潜在的对网络安全造成不良影响的事件，包括自然灾害、非恶意人为损坏和网络攻击

狭义网络安全威胁：指各类网络攻击行为

网络危险的原因：

技术因素

协议缺陷：TCP/IP 协议簇是互联网使用的标准协议集，然而这个协议簇在设计之初没有考虑安全问题，因此存在各种容易被利用的安全漏洞。具体而言，网络协议缺乏认证、加密等基本安全特性。

软件漏洞：信息系统中，几乎所有的设计都依赖软件实现。软件规模庞大，复杂度提高，开发者安全知识缺乏导致了各种软件安全问题

安全策略：安全需求和应用需求不一致，安全策略设计必须在安全和应用之间做出平衡，设计不当的安全策略会导致网络出现危险

硬件漏洞：硬件设计软件化使得软件中出现的漏洞同样会出现在硬件中。

人为因素：

攻击者掌握技术和资源

广大网络应用人群缺少安全知识，安全人才匮乏。

1.2 掌握

1.2.1 网络攻击标准以及分类

- 互斥性（分类类别不应重叠）
- 完备性（覆盖所有可能的攻击）
- 非二义性（类别划分清晰）
- 可重复性（对一个样本多次分类结果一致）
- 可接受性（符合逻辑和直觉）
- 实用性（可用于深入研究和调查）

按攻击发生时，攻击者与被攻击者之间的交互关系进行分类

1. 物理攻击 (local attack)：指攻击者通过实际接触被攻击主机实施的各种攻击方法

2. 主动攻击 (server-side attack)：指攻击者利用 web、FTP、telnet 等开放网络服务对目标实施的各种攻击。

3. 被动攻击 (client-side attack)：攻击者利用浏览器、邮件接收程序、文字处理程序等客户端应用程序漏洞或系统用户弱点，对目标实施各种攻击。

这里的主动攻击与被动攻击指网络攻击中的主动被动，不是泛指。

4. 中间人攻击 (man-in-middle attack)：攻击者处于被攻击主机的某个网络应用的中间人位置，进行数据窃听、破坏、篡改等攻击。

1.2.2 网络攻击步骤（5步）

网络攻击分为信息收集、获取权限、安装后门、扩大影响、清除痕迹五步

信息收集：尽可能多地收集目标的相关信息，为后续的“精确”攻击建立基础
主动攻击

利用公开信息服务

主机扫描与端口扫描

操作系统探测与应用程序类型识别

获取权限：获取目标系统的读、写、执行等权限

主动攻击：

口令攻击

缓冲区溢出

脚本攻击

被动攻击：

木马

使用邮件、IM 等发送恶意链接

安装后门：在目标系统中安装后门程序，以更加方便、更加隐蔽的方式对目标系统进行操控

实现方式：

主机控制木马

web 服务器控制木马

扩大影响：以目标系统为“跳板”，对目标所属网络中的其他主机进行攻击，最大程度扩大攻击效果

嗅探、假消息攻击等

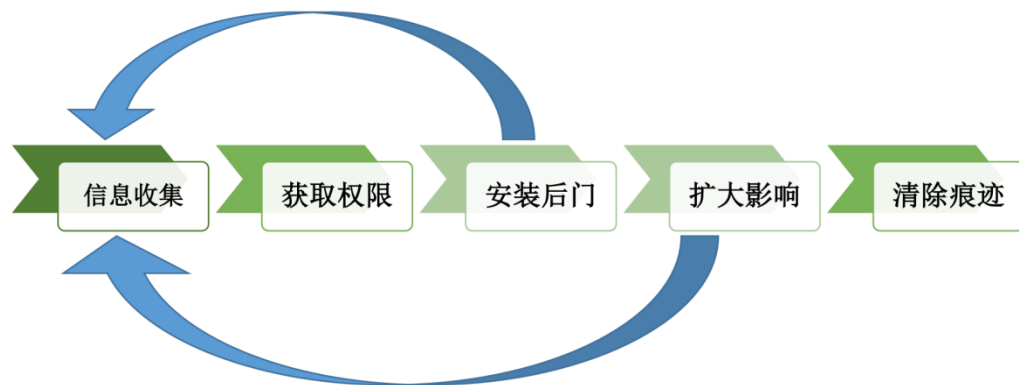
清除痕迹：清除攻击的痕迹，以尽可能长久地对目标进行控制，并防止被识别、追踪

主要方法：

Rootkit 隐藏

系统安全日志清除

应用程序日志清除



1. 2. 3 物理攻击和社会工程学

物理攻击：通过各种技术手段绕开物理安全防护体系，从而进入受保护的设施场所或设备资源内，获取或破坏信息系统物理媒体中受保护信息的攻击方式

社会工程学：利用人类的愚蠢，操纵他人执行预期的动作或泄漏机密信息的一门艺术与学问。

2. 第二章 信息收集技术

2. 1 了解

2. 1. 1 网络拓扑探测

通过对一个局域网的扫描，来实现对该局域网结构局部或全部的检测。

拓扑探测技术：

traceroute 技术：用来发现实际的路由路径

SNMP 技术：简单网络管理协议，专门设计用于在 IP 网络管理网络节点的一

种标准协议。通过 snmp 可以获取网络中的设备信息

2.2 掌握

2.2.1 公开信息收集定义，内容，分类和必要性

定义：信息收集是指黑客为了更加有效地实施攻击而在攻击前或攻击过程中对目标的所有探测活动。

目的：信息收集是尽可能多地收集攻击目标的相关信息，其目的是为后续的“精确”攻击建立基础

内容：

域名、IP 地址、Mac 地址

防火墙、IDS 等安全措施信息

目标所处内部网络结构、域组织架构

目标硬件信息、操作系统类型、开放端口信息

目标敏感文件目录、应用程序信息（和应用程序漏洞）

分类：

信息收集方法分为主动和被动

主动信息收集：通过直接访问、扫描网站等技术实现，特点是攻击者流量经过网站

主动信息收集方法包括网路扫描、漏洞扫描和网络拓扑扫描

被动信息收集：利用第三方服务对目标进行访问了解，攻击者流量不经过网站。

被动信息收集方法包括：利用 web 服务，利用搜索引擎、利用 WhoIS 服务，利用 DNS 服务和社会工程学

必要性：

攻击者：先手优势 攻击目标信息收集

防御者：后发制人？ 对攻击者实施信息收集，归因溯源

2.2.2 网络扫描类型以及原理

主机扫描：通过向目标主机发送网络流量包，来获取主机的信息。

Ping：通过 ICMP 报文的响应结果，来判断主机的信息。ping 采用 8 类型的报文，可以判断主机是否在线。

名称	类型
ICMP Destination Unreachable（目标不可达）	3
ICMP Source Quench（源抑制）	4
ICMP Redirection（重定向）	5
ICMP Timestamp Request/Reply（时间戳）	13/14
ICMP Address Mask Request/Reply（子网掩码）	17/18
ICMP Echo Request/Reply（响应请求/应答）	8/0

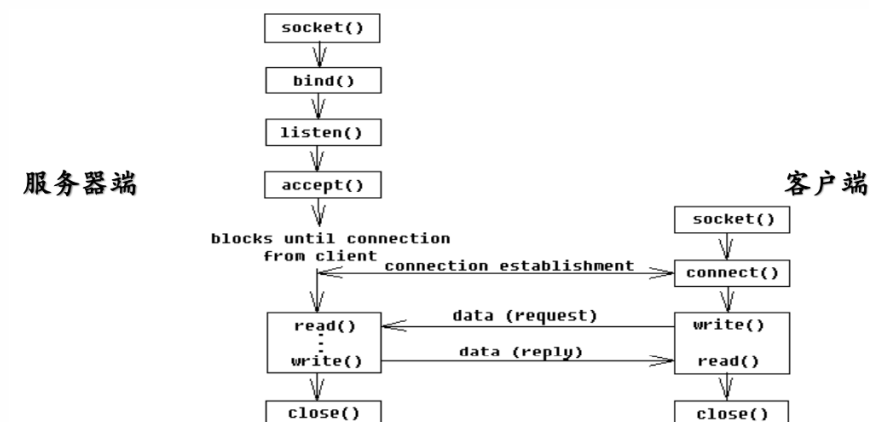
高级 IP 扫描技术：即构建异常 IP 头来观察主机返回信息，包括在 IP 头中设置无效字段和错误的数据分片

设置 IP 头无效字段：在 IP 头设置无效字段值，如 Header Length Field 和 IP options Field，主机或过滤设备会反馈 ICMP Parameter Problem Error 信息。

错误的数据分片：向目标主机发送的 IP 包中填充错误的 seq 字段，目标主机返回 ICMP destination Unreachable 信息

端口扫描：端口是通信的通道，是主机和外部网络交流出入口，也是我们进行攻击的入口。端口分为 TCP 端口和 UDP 端口，因此端口扫描可分类为 TCP 扫描和 UDP 扫描

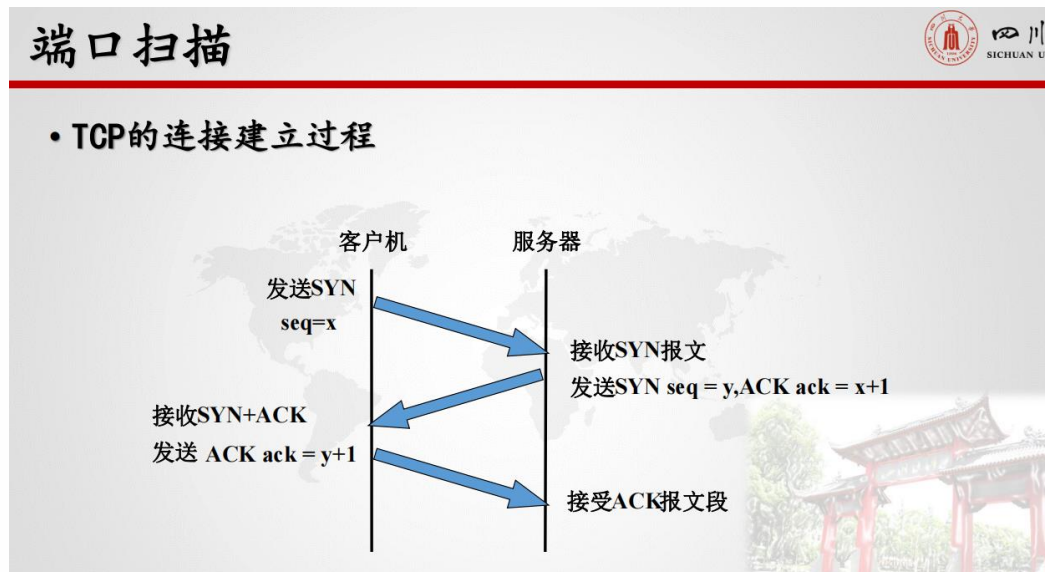
TCP connect 端口扫描：



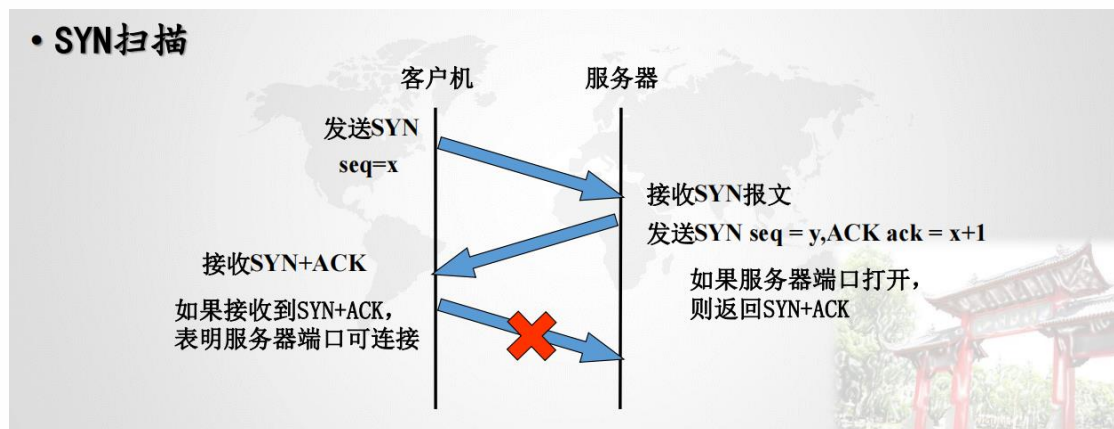
如果端口开放，则可以建立连接。

优点:实现简单，可以以普通用户权限执行

缺点：连接会被目标的应用日志记录



TCP SYN:



优点：一般不会被目标主机记录

缺点：发送和接受 SYN 包需要 raw socket，开启 raw socket 必须有管理员权限。

TCP FIN: 直接发送 FIN 报文，打开的端口会忽略，关闭的端口会返回 RST 报文。因此也可以通过 TCP FIN 构建

优点：不会被记录到日志，也可以绕过某些防火墙，netstat 命令不会显示

缺点：需要 Raw IP 编程，实现起来稍微复杂；不同操作系统返回可能不同，不完全可信。

端口扫描 - 半连接扫描



- ①半连接扫描，扫描者向目标主机发送一个SYN，如果目标主机回复了一个SYN/ACK数据包，那么说明主机存活，如果收到一个RST/ACK数据包，那么主机没有存活。
- 因为扫描者只向目标主机发送了SYN，并没有和目标主机进行连接，因此称为半连接。

端口扫描 - ACK



- ②ACK，设置flags位为ACK，不回复表示端口关闭或被过滤，如果回复的数据包TTL小于等于64表示端口开放，大于64端口关闭(windows)。

端口扫描 - FIN



- ③FIN，FIN扫描和NULL扫描类似，将标志位FIN置1，如果端口开放，则没有反应，端口关闭，目标主机会发送RST。

端口扫描 - Null



- ④Null，设置flags位为空，不回复则表示端口开启，回复并且回复的标志位为RS表示端口关闭

⑤Xmas, 设置所有标志位, 不回复则表示端口开启, 回复所有标志位表示端口关闭

端口扫描常用工具: Nmap, SuperScan

系统类型扫描: 可以通过端口扫描结果分析、banner 和 TCP/IP 协议指纹来判断

端口扫描结果分析:

由于每个操作系统都有其特有的开启的端口, 因此可以通过端口扫描结果分析出操作系统:

Windows 9X: 137, 139

Windows 2000/XP: 135, 139, 445

Unix: 512-514, 2049

应用程序 banner: 服务程序接收到客户端正常连接后, 会在欢迎信息中给出操作系统类型

TCP/IP 协议栈指纹: 不同操作系统实现 TCP/IP 协议时都会有些差异, 我们称这些差异为 TCP/IP 指纹。(如何使用不需要掌握)

协议栈指纹探测工具: Checkos, Queso, Nmap

2.2.3 漏洞扫描目的, 原理, 组件和方法

漏洞扫描: 指利用一些专门或综合漏洞扫描程序对目标存在的系统漏洞或应用程序漏洞进行扫描。

计算机安全领域, 安全漏洞通常又称作脆弱性。

漏洞扫描的缺陷: 报告不一定可靠, 且易暴露目标。

漏洞扫描工具有: Nessus, ISS, SATAN/SAINT

漏洞检测: 对重要计算机信息系统进行检查, 发现其中可以被黑客利用的漏洞。漏洞检测有 2 种策略 3 种方法。

策略为：被动式策略和主动式策略

被动式策略是基于主机的检测，对系统中不合适的设置、脆弱口令以及与安全规则相抵触的对象进行检查

主动式策略式基于网络的检测，通过执行一些脚本文件对系统进行攻击，记录反应从而发现漏洞（网络模糊测试）。

漏洞检测的主要方法由直接测试、推断和带凭证的测试

直接测试是利用漏洞特点发现系统漏洞的方法

推断是不直接渗透漏洞，只是间接地寻找漏洞存在证据的方法

带凭证的测试：凭证是指访问服务需要的用户名或密码，带凭证的测试是指赋予测试人员目标系统中的角色，从而更好地模拟渗透，来发现存在漏洞。

2.2.4 网络拓扑探测

拓扑探测

Traceroute 技术：用来发现实际的路由路径

SNMP：不同类型网络设备之间客户机/服务器模式的简单通信协议。

两个基本命令模式：

- Read：观察设备配置信息。
- Read/Write：有权写入信息。

网络设备(路由器、交换机) 识别

- 搜索引擎：Shodan、ZoomEye
- 基于设备指纹的设备类型探测

Banner 信息的获取渠道：

- FTP 协议
- SSH
- Telnet
- HTTP

网络实体 IP 地理位置定位

- 基于查询信息的定位

通过查询机构注册的信息确定网络设备的地理位置；

- 基于网络测量的定位

利用探测源与目标实体的时延、拓扑或其他信息估计目标实体的位置

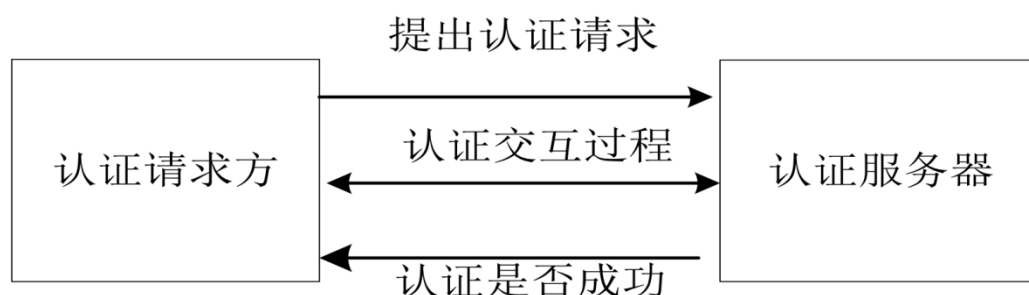
3. 第三章 口令攻击

3.1.1 口令的定义和作用

用户向计算机系统以一种安全的方式提交自己的身份证明，然后由系统确定用户的身份是否属实，最终拒绝用户或者赋予用户一定权限。

口令是身份认证的一种方式

身份认证过程



口令攻击是黑客通过获取系统管理员或其他殊用户的口令，获得系统的管理权，窃取系统信息、磁盘中的文件甚至对系统进行破坏。

口令攻击分为针对口令强度的攻击，针对口令存储的攻击和针对口令传输的攻击

3.1.2 针对口令强度的攻击

针对口令强度的攻击方式：字典攻击，强力攻击，组合攻击，撞库攻击和彩虹表攻击

- 字典攻击：使用概率较高的口令，集中存放在字典文件中，通过不同的变异规则猜测字典
- 强力攻击：用速度足够快的计算机常识字母、数字、特殊字符的所有组合，来破解口令
- 撞库攻击：攻击者透过手机网上已泄露的用户名与口令，用这些账户和口令尝试登陆其他网站。如果用户使用相同的用户名和密码，那么就可以实现攻击
- 彩虹表攻击：一种破解哈希算法的技术
- 组合攻击：上述攻击的组合使用

3.1.3 针对口令存储的攻击

口令存储：Windows 口令存储机制——SAM 数据库：安全账户管理器

针对口令存储的攻击：可以获取硬盘上系统保存的口令字、直接读取 Windows 系统存储在内存中的登录口令或者破解 SAM 来实现。

3.1.4 针对口令传输的攻击

针对口令传输的攻击：可以通过嗅探攻击、键盘记录，网络钓鱼和重放攻击来实现

3.1.5 强弱口令

强口令：不容易被发现规律，且有足够的长度。对长度的要求随应用环境的不同而不同，应该使得攻击者在某个时间段内很难破解

弱口令：容易被发现规律或长度不足的口令。

3.1.6 口令攻击的防范

不要在所有系统中使用相同的口令，并定期更换口令

重要的账户使用强口令，不重要的账户可以降低要求

设置口令安全策略，规定计算机对口令的定期清除

采用加密的通信协议来传输口令

访问网站时注意区分网站是否为虚假网站

定期查杀木马

4. 第四章 软件漏洞

4.1 了解

4.1.1 漏洞概念

漏洞：指信息系统硬件、软件、操作系统、网络协议和数据库等在设计上、是线上出现的可以被攻击者利用的错误、缺陷和疏漏

漏洞攻击 3 步骤：漏洞发现、漏洞分析、漏洞利用

漏洞发现：发现潜在的程序代码漏洞和系统安全机制漏洞

漏洞分析：通过程序执行调试与上下文环境分析，定位并确定漏洞，记录漏洞发生的执行过程

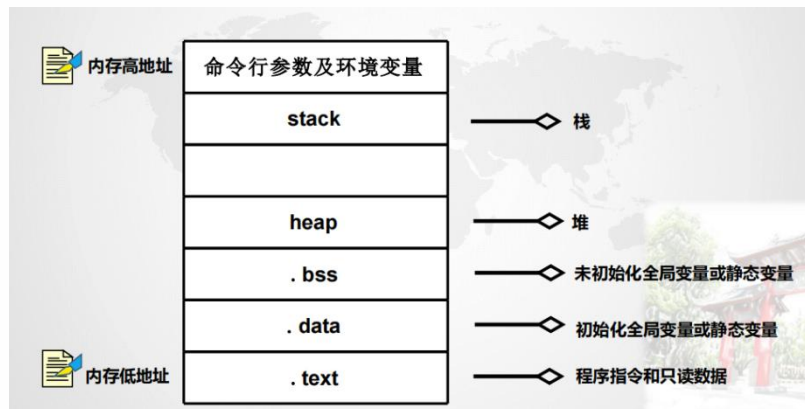
漏洞利用：通过**攻击源**、**有效载荷构造**实现系统漏洞的利用和保护机制的突破

漏洞导致后果：以匿名身份获取系统最高权限、被植入后门、实施远程拒绝服务攻击

4.1.2 典型漏洞类型：栈溢出、堆溢出、格式化串、整型溢出、释放再使用

4.1.3 栈溢出漏洞利用原理

内存分布：



漏洞利用内存变化：

在局部变量的下面，是前一个调用函数的 EBP，接下来就是返回地址。

如果局部变量发生溢出，很有可能会覆盖掉 EBP 甚至 RET(返回地址)，这就是缓冲区溢出攻击的“奥秘”所在。



压栈/出栈：

栈（Stack）是一种用来存储函数调用时的临时信息的结构，如函数调用所传递的参数、函数的返回地址、函数的局部变量等。

PUSH 操作：向栈中添加数据，称为压栈，数据将放置在栈顶；

POP 操作：POP 操作相反，在栈顶部移去一个元素，并将栈的大小减一，称为弹栈

栈溢出原理：

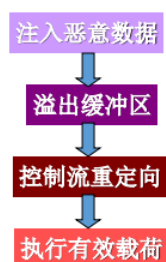
□ 程序中发生函数调用时，计算机做如下操作：

- 首先把指令寄存器EIP（它指向当前CPU将要运行的下一条指令的地址）中的内容压入栈，作为程序的返回地址（下文中用RET表示）；
- 之后放入栈的是基址寄存器EBP，它指向当前函数栈帧（stack frame）的底部；
- 然后把当前的栈指针ESP拷贝到EBP，作为新的基地址，最后为本地变量的动态存储分配留出一定空间，并把ESP减去适当的数值。

如果在堆栈中压入的数据超过预先给堆栈分配的容量时，就会出现堆栈溢出，从而使得程序运行失败；如果发生溢出的是大型程序还有可能会导致系统崩溃。

4.1.4 溢出漏洞利用

基本流程：



溢出点定位：

探测法：构造数据，根据出错的情况来判断。

反汇编分析

覆盖执行控制地址：

执行控制地址可包括：

- 覆盖返回地址
- 覆盖函数指针变量
- 覆盖异常处理结构

覆盖异常处理结构：

• 异常处理是一种对程序异常的处理机制，它把错误处理代码与正常情况下所执行的代码分开。

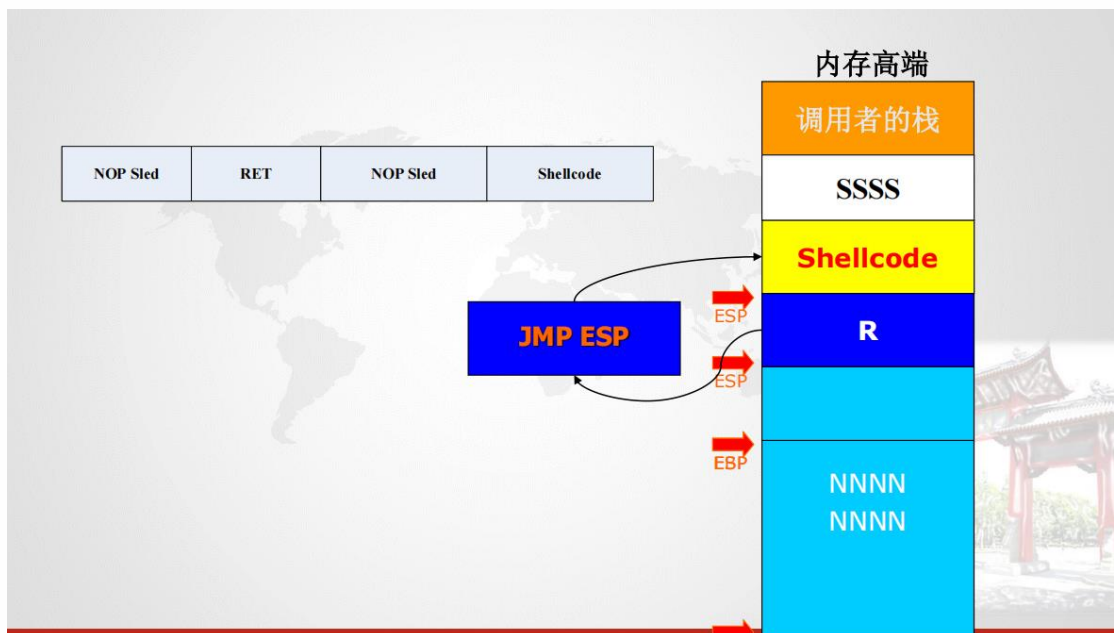
- 当程序发生异常时，系统中断当前线程，将控制权交给异常处理程序

- Windows 的异常处理机制称为结构化异常处理（Structured Exception Handling）

跳转地址的确定：

- 跳转指令的选取
- jmp esp、call ebx、call ecx 等。
- 跳转指令的搜索范围
- 用户空间的任意地址、系统 dll、进程代码段、PEB、TEB
- 跳转指令地址的选择规律

Shellcode 的定位和跳转：



4.1.5 Shellcode

ShellCode 就是一段能够完成一定功能(比如打开一个命令窗口)、可直接由计算机执行的机器代码，通常以十六进制的形式存在。

作用：

- 可以是处于恶作剧弹出对话框
- 打开 dos 窗口
- 添加系统管理用户
- 打开可以远程连接的端口
- 发起反向连接
- 上传（下载）木马病毒并运行
- 可能是攻击性的，删除重要文件、窃取数据
- 破坏，格式化磁盘

编写 shellcode:

- 编写打开 windows 对话框的 C 程序
- Windows 函数调用原理
- 使用汇编生成 ShellCode

注意事项:

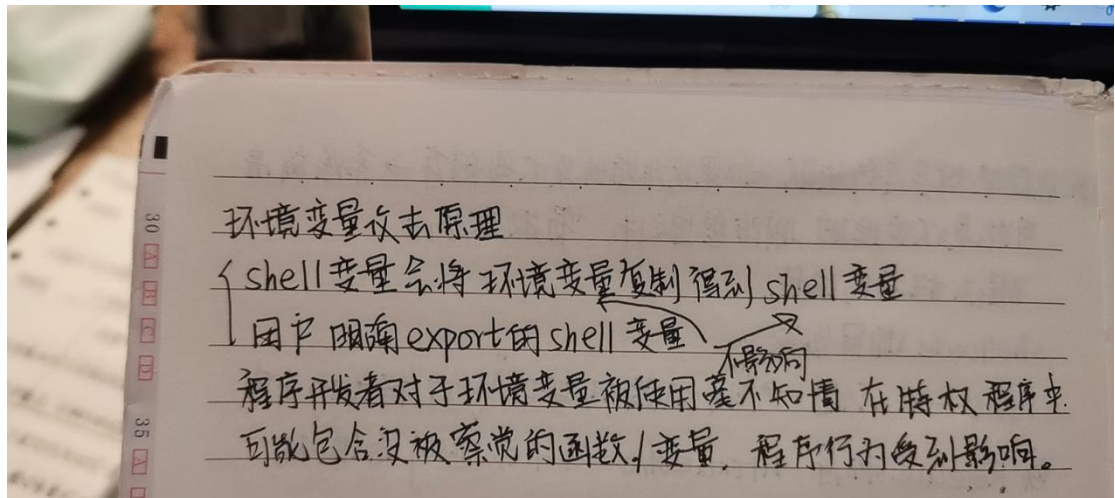
- 1) shellcode 调用 MessageBox 函数后，没有做扫尾工作。
- 2) Shellcode 包含 null 字节。
- 3) 如果 user32.dll 没有加载时，那个 API 地址将不会指向 MessageBoxA 函数，代码将会失败。
- 4) Shellcode 包含了一个 MessageBoxA 函数地址的硬编，不通用

通用 ShellCode 的编写方法:

- ✓ 将每个版本的 Windows 操作系统所对应的函数的地址列出来，然后针对不同版本的操作系统使用不同的地址；
- ✓ 动态定位函数地址：使用 GetProcAddress 和 LoadLibrary 函数动态获取其它函数的地址。

4.1.6 环境变量攻击

原理:



□ 进程可以通过以下两种方式来获取环境变量：

- ✓ 如果使用fork()创建了一个新进程，则子进程将继承其父进程的环境变量。
- ✓ 如果进程使用execve()启动一个新程序，在此场景中内存空间被覆盖，所有旧环境变量将丢失。可以以一种特殊的方式调用执行lost.Execve()，以将环境变量从一个进程传递给另一个进程。

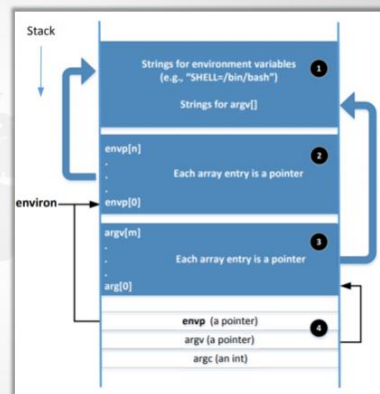
□ 调用 execve() 来传递环境变量：

```
int execve(const char *filename, char *const argv[],
           char *const envp[])
```

环境变量的内存位置



- envp和environ最初指向同一个地方
- envp只能在主函数中可访问，而environ是一个全局变量
- 当对环境变量进行更改时（例如添加新变量），存储环境变量的位置可能会移动到堆中，因此environ将发生更改（envp不会更改）



Set-uid:

补充：Set-UID 概念



- 允许用户以程序所有者的权限运行程序。
- 允许用户以临时提升的权限运行程序
- 示例： passwd 程序

```
$ ls -l /usr/bin/passwd  
-rwsr-xr-x 1 root root 41284 Sep 12 2012 /usr/bin/passwd
```

- 每个进程都有两个用户ID。
- Real UID (RUID):确定进程的真正所有者
- Effective UID (EUID): 标识进程的权限
 - ✓ 访问控制基于EUID
- 当执行正常程序时, $RUID = EUID$, 它们都等于运行程序的用户的ID
- 当执行Set-UID时, $RUID \neq EUID$. RUID还是用户 ID, 但是 EUID 是程序 owner 的 ID.
 - ✓ 如果程序归root所有, 则程序以root权限运行。

Set-UID方法VS服务方法



- 大多数操作系统遵循两种方法, 允许正常用户执行特权操作
 - ✓ Set-UID方法: 普通用户必须运行一个特殊的程序才能临时获得根权限
 - ✓ 服务方法: 普通用户必须请求特权服务才能为他们执行操作。前面的幻灯片中的图描述了这两种方法
- Set-UID具有更广泛的攻击面, 这是由环境变量引起的
 - ✓ 在Set-UID方法中不能信任环境变量
 - ✓ 环境变量在服务方法中可以被信任
- 尽管其他攻击面仍然适用于服务方法 (在第1章中讨论), 但它被认为比Set-UID方法更安全
- 因此, 安卓操作系统完全删除了Set-UID和Set-GID机制

5. 第五章 Web 应用攻击

5.1.1 Web 应用基本概念

Web 应用程序是一种可以通过 Web 访问的应用程序

web 应用的核心要素包括 web 服务器，web 客户端和 HTTP 协议

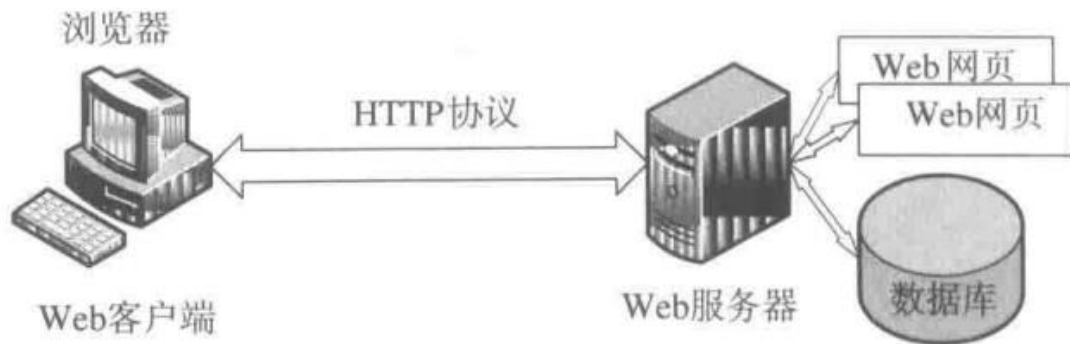


图 5-1 Web 应用基本原理

目前最常见的 B/S 架构由 web 网页，web 服务器，浏览器和 http 协议构成

web 网页：web 网页位于 web 服务器上，用于展示信息，一般采用 HTML 编写。

web 服务器：主要功能是以 web 网页形式提供网上信息浏览服务，当用户访问的网页不存在时，返回 404 not found 错误

浏览器：浏览器用户从 web 服务器上获取信息，并根据信息格式展示信息内容，是用户使用 web 应用程序的必备应用软件。

HTTP 协议：http 协议用于 web 客户端和 web 服务器之间的信息交互，采用请求/响应模式，包括请求/响应两种报文。

5.1.2 web 应用攻击类型

web 攻击类型分为 web 客户端攻击、web 服务器攻击和 http 协议攻击

web 客户端攻击：该类攻击的目标是访问 web 服务器的系统或用户。最典型的攻击是跨站脚本攻击（XSS 攻击）

web 服务器攻击：该类攻击的目标是 web 服务器，典型的攻击方式有网页篡改、代码注入和文件操作控制攻击。SQL 攻击是 web 服务器攻击的一种

http 协议攻击：该类攻击的目标是 HTTP 的相关数据，典型攻击方式有 HTTP 头注入攻击和 HTTP 会话攻击

5.1.3 Xss 攻击

XSS 攻击是由于 Web 应用程序对用户输入过滤不足而产生的，使得攻击者输入的特定数据变成了 JavaScript 脚本或 HTML 代码

同源策略：它的含义是指，A 网页设置的 Cookie，B 网页不能打开，除非这两个网页“同源”。所谓“同源”指的是“三个相同” 协议相同 域名相同 端口相同

危害：(1)网络钓鱼，包括盗取各类用户账号

(2)窃取用户 cookies 资料，从而获取用户隐私信息，或利用好用户身份进行一部对网站执行操作

(3)劫持用户（浏览器）会话，从而执行任意操作，例如非法转账、强制发表日志、发送电子邮件等

(4)强制弹出广告页面、刷流量等；

(5)网页挂马

(6)进行恶意操作，例如任意篡改页面信息、删除文章等；

(7)进行大量的客户端攻击，如 DDoS 攻击；

(8)提取客户端信息，例如用户的浏览历史、真实 IP、开放端口等；

(9)控制受害者机器向其它网站发起攻击；

(10)结合其他漏洞，如 CSRF 漏洞，实施进一步作恶；

(11)提升用户权限，包括进一步渗透网站；

(12)传播跨站脚本蠕虫；

类型：

XSS 根据其特性和利用手法的不同，主要分成三大类：

■ 反射型 XSS：非持久性、参数型跨站脚本

恶意脚本附加到 URL 地址参数中

■ 存储型 XSS：持久型

一般攻击存在留言、评论、博客日志等中

恶意脚本被存储在服务端的数据库中

■ DOM 型 XSS：DOM XSS 是基于在 js 上的

不需要与服务端进行交互

网站有一个 HTML 页面采用不安全的方式：

`document.location`

漏洞利用方式：

Cookie 窃取

会话劫持：指攻击者通过利用 XSS 攻击，冒用合法者的会话 ID 进行网络访问的一种攻击方式。

网络钓鱼：攻击者可以执行 JavaScript 代码动态生成网页内容或直接注入 HTML 代码，从而产生网络钓鱼攻击。

信息刺探：利用 XSS 攻击，可以在客户端执行一段 JavaScript 代码，因此：

- 攻击者可以通过这段代码实现多种信息的刺探
- 访问历史信息、端口信息、剪贴板内容、客户端 IP 地址、键盘信息等。

网页挂马：将 Web 网页技术和木马技术结合起来就是网页挂马。

- 攻击者将恶意脚本隐藏在 Web 网页中，当用户浏览该网页时，这些隐藏的恶意脚本将在用户不知情的情况下执行，下载并启动木马程序。

Xss 蠕虫：XSS 蠕虫是指利用 XSS 攻击进行传播的一类恶意代码，

- 一般利用存储型 XSS 攻击。
- XSS 蠕虫的基本原理就是将一段 JavaScript 代码保存在服务器上，其他用户浏览相关信息时，会执行 JavaScript 代码，从而引发攻击。

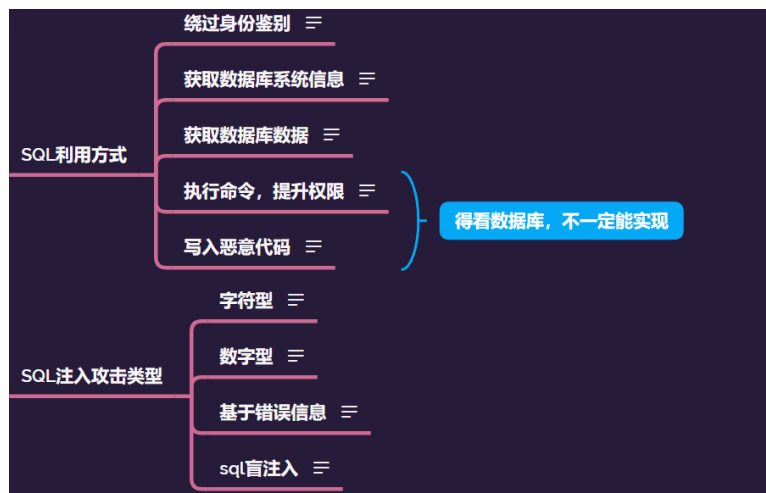
防范措施：

HttpOnly 属性：HttpOnly 是另一个应用给 cookie 的标志，而且所有现代浏览器都支持它。HttpOnly 标志的用途是指示浏览器禁止任何脚本访问 cookie 内容，这样就可以降低通过 JavaScript 发起的 XSS 攻击偷取 cookie 的风险。

安全编码：PHP 语言中针对 XSS 攻击的安全编码函数有 `htmlspecialchars` 和 `htmlspecialchars` 等

5.1.4 Sql 注入攻击

就是向网站提交精心构造的 SQL 查询语句，导致网站将关键数据信息返回



注入步骤:

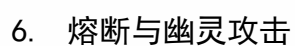
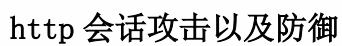
1. 注入点的发现
2. 数据库的类型
3. 猜解表名
4. 猜解字段名
5. 猜解内容
6. 进入管理页面，上传 ASP 木马

提权方法:

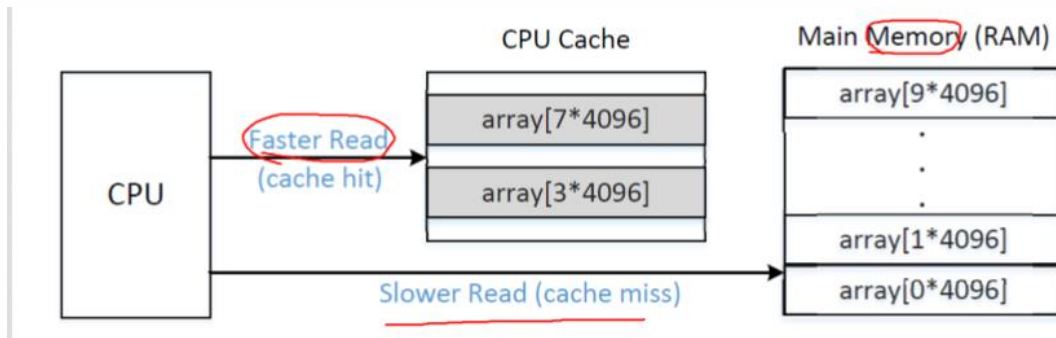
木马上传成功以后，我们已经初步的得到了一个 webshell，但是只是 user 权限，我们的最终目的是 syetem 权限，因此我们有必要对 webshell 进行提权，webshell 提权的方法也有很多，这里只举几个常用的例子。

- 最简单的提权方法是如果服务器上有装了 pcanywhere 服务端，管理员为了管理方便，一般的服务器都装了这个远程控制软件也给了我们方便，到系统盘的 DocumentsandSettings/AllUsers/Application Data/Symantec/pcAnywhere/中下载*.cif 本地 破解 cif 文件后，使用 pcanywhere 连接就完全控制服务器了。
- 利用 servu 来提升权限，servu 是一个非常好用的 ftp 服务器。
- 下载服务器 c: \winnt\system32\config 下的 sam 文件，得到后在本地进行破解，等到服务器的管理员的用户名和密码。
- 脚本提权 c:\Documents and Settings\All Users\「开始」菜单\程序\启动"写入 bat, vbs

下载到本地



Cpu 缓存原理:



侧通道攻击原理：

目前，绝大多数计算机均在CPU和内存之间增加CPU缓存（Cache），采用这种体系结构可以显著地提高程序的平均执行性能。然而，如果CPU访问Cache中并不存在的数据时，则将会产生时间延迟，因此此时目标数据必须重新从内存加载到Cache中。测量这种时间延迟有可能让攻击者确定出Cache访问失败的发生和频率。这就是基于缓存的侧信道攻击的基本原理。

熔断攻击思路：

☒ 熔断攻击思路

a. mov rax (rdx) b. add rax \$123 c. mov rcx (rbx)

CPU采用乱序执行的方式，cpu在执行a语句的同时将c语句中的内存位置的信息加载到cache中，这个加载的过程忽略了权限的判断，所以此时cache中有：(rdx)、(rbx)。之后再执行b语句、c语句，在执行c语句时发现并没有权限，所以放弃乱序执行的结果，回滚到最初的状态依次执行。此时cache中的信息没有还原。之后我们通过边信道攻击来测试数据，发现对于某一个数据的访问要远远快于其他数据的访问（因为访问cache中的数据要快于访问内存中的数据），那么就可以知道该数据在cache中，并且可以反推回它的内存地址。

幽灵攻击思路：

☒ 幽灵攻击思路

if (a>b) {c = array[123456789]}

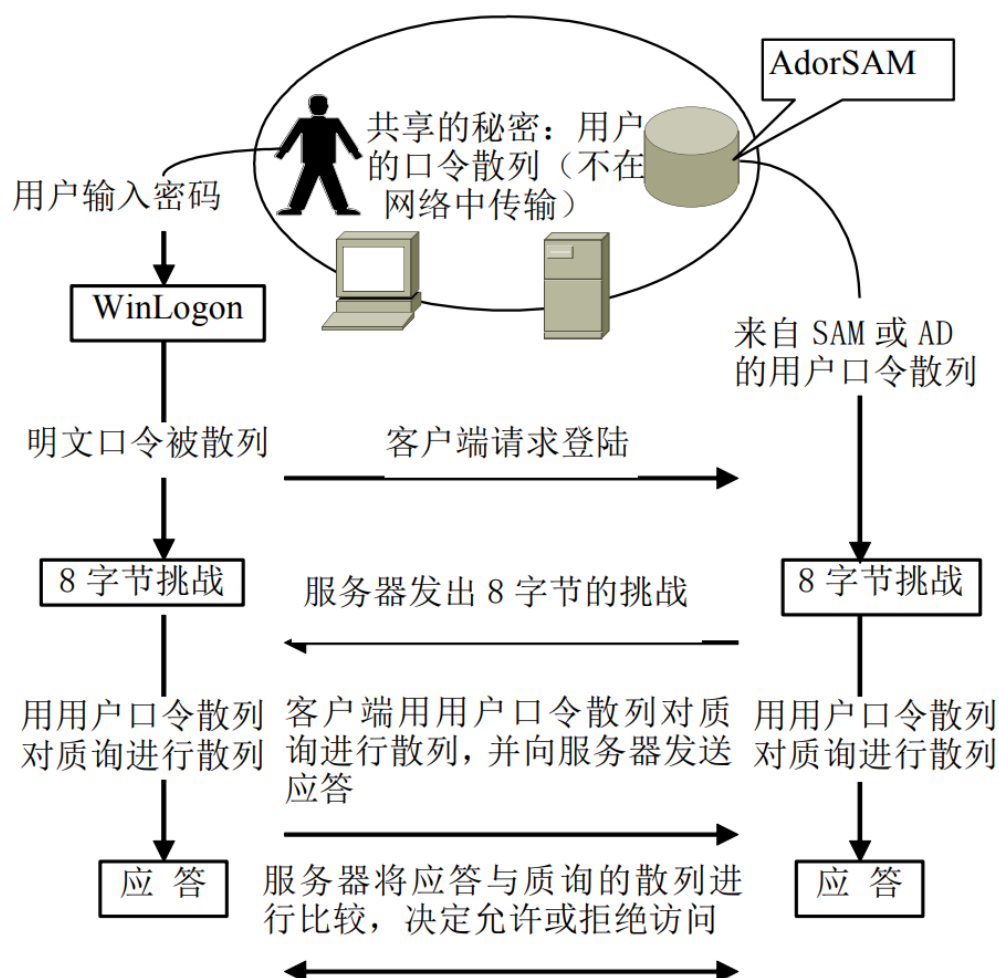
在执行if判断之前，cpu先将array[123456789]的数据加载到cache中，之后进行判断，发现a<b，这样cpu回滚到原来的状态，但是cache中的信息并没有恢复。之后与熔断一样，通过边信道攻击来测试数据，发现对于某一个数据的访问要远远快于其他数据的访问（因为访问cache中的数据要快于访问内存中的数据），那么就可以知道该数据在cache中，并且可以反推回它的内存地址。

7. 其他知识点

1) VPN 主要采用 4 项技术来保证安全，这 4 种技术分别是隧道技术、加解密技

术、密钥管理技术和身份认证技术。

2) Windows 的 NTLM 身份验证过程：质询/应答机制的应用



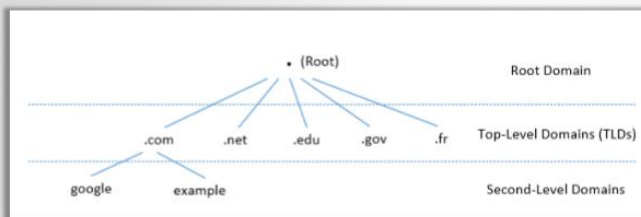
- ## 3) 管道 (Pipe)：是一种简单的进程间通信 (IPC) 机制，一个进程往管道中写入数据，一个进程从管道中读取数据。管道分为命名管道和匿名管道
- 命名管道：可以在同台设备不同进程或不同机器上的进程间进行**双向通信**
- 匿名管道：只能在父子进程间或者两个子进程间进行**单向通信**。

DNS 攻击（域名结构、查询过程、DNS 攻击类型及原理（本地 DNS 缓存中毒攻击、 远程 DNS 缓存中毒攻击、恶意 DNS 服务器的回复伪造攻击、DNS 重绑定攻击）、防范措施）

TCP协议



- 传输控制协议（TCP）是Internet协议套件的核心协议。
- 位于IP层的顶部；传输层。
- 为应用程序提供主机到主机的通信服务。
- 两个传输层协议
 - TCP：在应用程序之间提供可靠且有序的通信通道。
 - UDP：具有较低开销的轻量级协议，可用于不需要可靠性或通信顺序的应用程序。



- 域名称空间以层次树状结构组织。

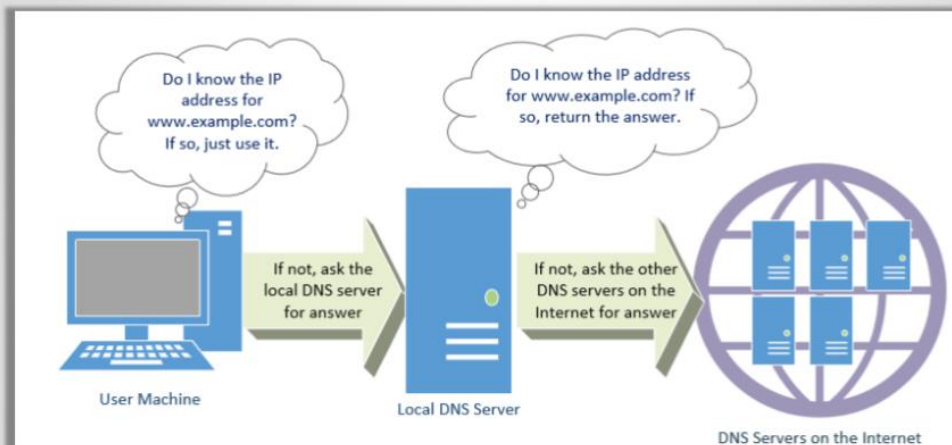
- 每个节点称为域或子域。

- 域的根称为根，表示为“.”。

- 在根目录下，我们有顶级域（TLD）。例如：在 **www.example.com** 中，TLD是.com。
- 域层次结构的下一级是第二级域，通常分配给特定实体，如公司、学校等



DNS查询过程



- 拒绝服务攻击 (DoS)：当本地DNS服务器和权威名称服务器不响应DNS查询时，计算机无法检索IP地址，这从本质上减少了通信。
- DNS欺骗：
 - 主要目标：向受害者提供欺诈性IP地址，诱使它们与不同于它们意图的机器进行通信。
 - 示例：如果用户打算访问银行网站进行网上银行业务，但通过DNS过程获得的IP地址是攻击者的机器，则用户机器将与攻击者的web服务器通信。

本地DNS缓存中毒攻击

```
$ dig www.example.net
; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.example.net
;; global options: +cmd
;; Got answer:
;; ->>HEADER<- opcode: QUERY, status: NOERROR, id: 61991
;; flags: qr aa ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 0

;; QUESTION SECTION:
;www.example.net.      IN A

;; ANSWER SECTION:
www.example.net.      259200 IN A      1.2.3.4      ①

;; AUTHORITY SECTION:
example.net.          259200 IN NS      ns.attacker32.com. ②
```

检查缓存

```
; authauthority
example.net.      259185 NS      ns.attacker32.com.
; authanswer
www.example.net.  259185 A      1.2.3.4
```

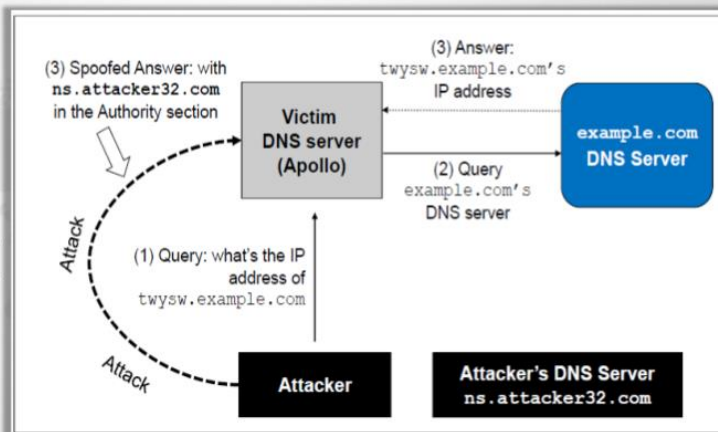
- 运行“`sudo rndc dumpdb -cache`”并检查“`/var/cache/bind/dump.db`”的内容。
- 在进行攻击之前，请使用“`sudo rndc flush`”清理缓存。

Kaminsky攻击

我们如何在不担心缓存效应的前提下不断伪造回复？

卡明斯基的想法：

- 每次询问不同的问题，因此缓存答案并不重要，并且本地DNS服务器每次都会发送一个新的查询。
- 在授权部分提供伪造答案



来自恶意DNS服务器的攻击

- 当用户访问网站（如attacker32.com）时，DNS查询最终将到达attacker32.com域的权威名称服务器。除了在响应的应答部分提供IP地址外，DNS服务器还可以在授权和其他部分提供信息。攻击者可以使用这些部分提供欺诈信息。

防止DNS缓存中毒攻击

•DNSSEC

- DNSSEC是DNS的一组扩展，旨在对DNS数据提供身份验证和完整性检查。
- 使用DNSSEC，来自DNSSEC保护区的所有答案都经过数字签名。
- 通过检查数字签名，DNS解析器能够检查信息是否真实。
- DNS缓存中毒将被此机制击败，因为将检测到任何虚假数据，因为它们将无法通过签名检查。

来自恶意DNS服务器的回复伪造攻击

•例子：

- 给定IP地址128.230.171.184，DNS解析器构造一个“假名称”184.171.230.128.in-addr.arpa，然后通过迭代过程发送查询。
- 我们在dig命令中使用@option模拟整个反向查找过程。

反向DNS查找



- 步骤1: 询问根服务器。我们得到in-addr.arpa区域的名称服务器。

```
seed@ubuntu:~$ dig @a.root-servers.net -x 128.230.171.184

;; QUESTION SECTION:
;184.171.230.128.in-addr.arpa.  IN PTR

;; AUTHORITY SECTION:
in-addr.arpa. 172800 IN NS f.in-addr-servers.arpa.
in-addr.arpa. 172800 IN NS e.in-addr-servers.arpa.

;; ADDITIONAL SECTION:
f.in-addr-servers.arpa. 172800 IN A 193.0.9.1
e.in-addr-servers.arpa. 172800 IN A 203.119.86.101
```

- 步骤2: 询问in-addr.arpa区域的名称服务器。我们为128.in-addr.arpa区域提供名称服务器

```
seed@ubuntu:~$ dig @f.in-addr-servers.arpa -x 128.230.171.184

;; QUESTION SECTION:
;184.171.230.128.in-addr.arpa.  IN PTR

;; AUTHORITY SECTION:
128.in-addr.arpa. 86400 IN NS r.arin.net.
128.in-addr.arpa. 86400 IN NS u.arin.net.
```

来自恶意DNS服务器的回复伪造攻击



- 步骤3: 询问128.in-addr.arpa区域的名称服务器。我们得到203.128.in-addr.arpa区域的名称服务器

```
seed@ubuntu:~$ dig @r.arin.net -x 128.230.171.184

;; QUESTION SECTION:
;184.171.230.128.in-addr.arpa.  IN PTR

;; AUTHORITY SECTION:
230.128.in-addr.arpa. 86400 IN NS ns2.syr.edu.
230.128.in-addr.arpa. 86400 IN NS ns1.syr.edu.
```

- 步骤4: 询问230.128.in-addr.arpa区域的名称服务器。我们得到了最终结果

```
seed@ubuntu:~$ dig @ns2.syr.edu -x 128.230.171.184

;; QUESTION SECTION:
;184.171.230.128.in-addr.arpa.  IN PTR

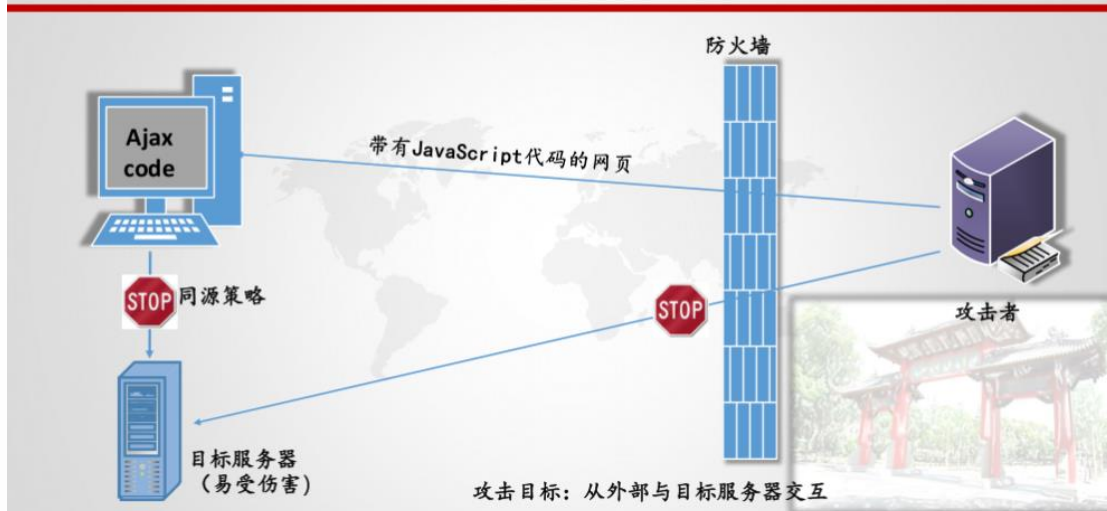
;; ANSWER SECTION:
184.171.230.128.in-addr.arpa. 3600 IN PTR syr.edu.
```

回顾我们的问题

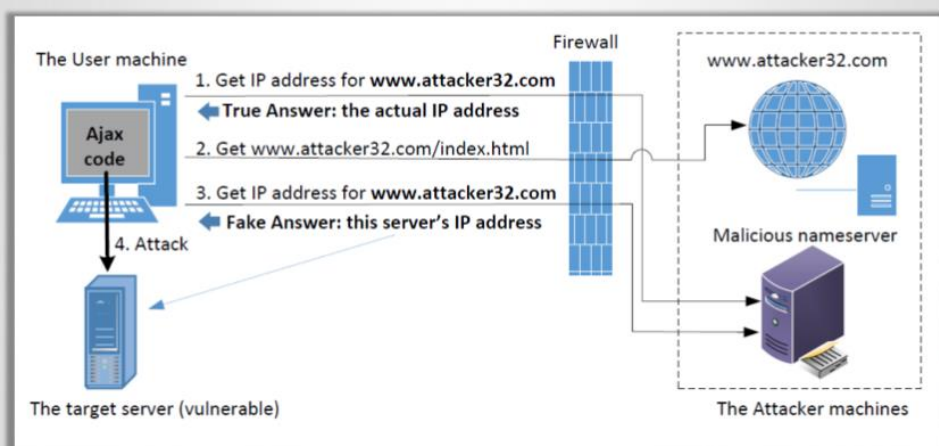


- 问题: 我们可以使用从反向DNS查找获得的主机名作为访问控制的基础吗?
- 答复:
 - 如果数据包来自攻击者, 则反向DNS查找将返回到攻击者的名称服务器。
 - 攻击者可以使用他们想要的任何主机名进行回复。

DNS重新绑定攻击



DNS重新绑定攻击

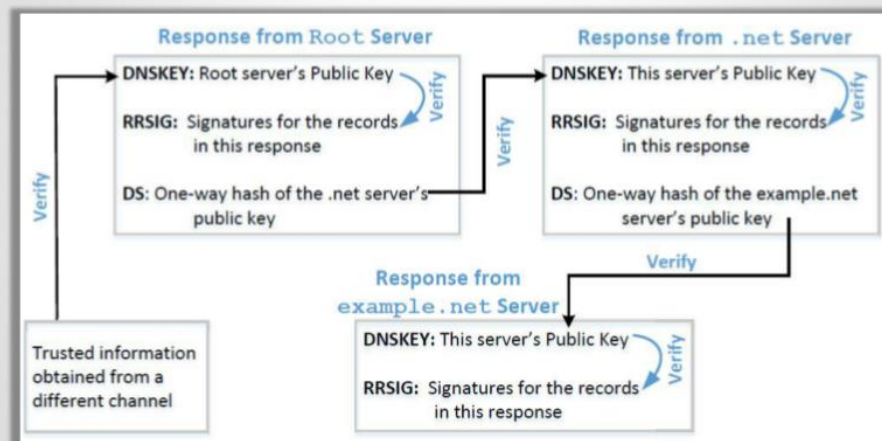


防止DNS缓存中毒攻击

•DNSSEC

- DNSSEC是DNS的一组扩展，旨在对DNS数据提供身份验证和完整性检查。
- 使用DNSSEC，来自DNSSEC保护区的所有答案都经过数字签名。
- 通过检查数字签名，DNS解析器能够检查信息是否真实。
- DNS缓存中毒将被此机制击败，因为将检测到任何虚假数据，因为它们将无法通过签名检查。

使用DNSSEC进行保护



使用TLS/SSL进行保护



- 传输层安全（TLS/SSL）协议提供了针对缓存中毒攻击的解决方案。
- 在使用DNS协议获取域名（`www.example.net`）的IP地址后，计算机将询问IP地址的所有者（服务器）是否为`www.example.net`。
- 服务器必须提供由受信任实体签名的公钥证书，并证明它知道与`www.example.net`关联的相应私钥（即，它是证书的所有者）。
- HTTPS构建在TLS/SSL之上。它可以击败DNS缓存中毒攻击。

DNSSEC与TLS/SSL比较



- DNSSEC和TLS/SSL都基于公钥技术，但它们的信任链不同。
- DNSSEC使用DNS区域层次结构提供信任链，因此父区域中的名称服务器为子区域中的名称服务器提供担保。
- TLS/SSL依赖于公钥基础设施，该基础设施包含为其他计算机提供担保的证书颁发机构。